



Common Problems

ChatGPT

⚡ Real-time Updates

🕒 Managing Long Running Tasks

By Evan King · Published May 21, 2026 · 🔥 hard

Understanding the Problem

💬 What is ChatGPT?

Unless you've been living under a rock, you know what ChatGPT is. It's a conversational AI product where users send prompts in natural language and get responses streamed back from a large language model. Conversations are saved, so users can come back to an old chat and pick up right where they left off.

For this problem we treat the LLM as a black box we call, not something we train or run the internals of. All the design lives in the serving system around it, in how we stream tokens back fast, how we schedule scarce GPUs, and how we keep cost sane as conversations grow. We'll also scope this to text in, text out only, with no images, audio, or video, and no editing or branching of existing messages.

Functional Requirements

Core Requirements

1. Users should be able to send a prompt in a chat and receive an AI-generated response.
2. Users should be able to view past chats and resume a conversation, with the chat's prior context carried into the prompt.

Below the line (out of scope)

- Editing or branching existing messages.
- Image, audio, or video input and output (text only).
- Sharing chats or collaborating on a chat with other users.
- Custom GPTs, tool / function calling, and web browsing.

- Full-text search across a user's chat history.

Non-Functional Requirements

Non-functional requirements cover the properties of the system that matter to the user and the business.

ChatGPT feels broken if you stare at a blank screen for a few seconds after hitting enter, so latency to the first token matters more than total completion time. Because GPUs are the scarce, expensive resource here, the system has to be deliberate about who gets compute and when. ChatGPT serves a little over 200M daily active users at the time of writing, so that's the scale we'll design against.

With that framing, here are the requirements that actually shape the design.

Core Requirements

1. The system should have low time-to-first-token ($< \sim 500\text{ms}$), with continuous, smooth streaming after that. A full response can take up to ~ 30 seconds to finish generating.
2. The system should prioritize high availability over strong consistency for conversation state ($\sim 99.9\%+$). It's better to return an error or a degraded experience than to block the whole system on perfectly synchronized chat state.
3. The system should scale under GPU-constrained capacity, with fair allocation across a tiered user base (200M DAU, $\sim 20\text{k}$ prompts/sec at peak, $\sim 120\text{k}$ concurrent in-flight streams).

Below the line (out of scope)

- Durability of every streamed token (we persist the final assistant message, not each chunk).
- Authentication, abuse prevention, and content moderation.
- GDPR, data residency, and privacy compliance.
- Monitoring, logging, alerting, and CI/CD.

Here's how it might look on your whiteboard:

Non-Functional Requirements

- Low TTFT (time-to-first-token) with continuous streaming
- High availability over strong consistency for chat state
- GPU-constrained scalability with fair allocation across tiers

Requirements



Adding features that are out of scope is a "nice to have". It shows product thinking and gives your interviewer a chance to help you reprioritize based on what they want to see. That said, it's very much a nice to have. If extra features aren't coming to you quickly, don't waste time, just move on.

The Set Up

Planning the Approach

Before designing anything, take a moment to plan. This is a product-style question, so we'll build the design up sequentially, going one by one through the functional requirements. There are only two of them, so the high-level design will be short, and that's intentional. The request path is almost boring, because the work that actually makes this problem interesting lives in the non-functional requirements, where we have to stream tokens back fast, schedule scarce GPUs, and keep cost under control. Those are what become our deep dives.

Defining the Core Entities

I like to begin with a broad overview of the primary entities. At this stage we don't need every column, just the nouns we'll reason about for the rest of the interview. We'll flesh out fields during the high-level design.

To satisfy our functional requirements, we'll need the following entities:

1. **User:** An account on the platform. Carries the tier (free vs paid), which is going to matter a lot once we get to fairness and scheduling.
2. **Chat:** A single conversation thread. Belongs to one user, groups an ordered sequence of messages, and carries a title and timestamps.

3. **Message:** One turn in a chat, either a user prompt or an assistant response. Carries the chatId, a role (`user` or `assistant`), the content, and a token count.

In the actual interview, this can be as simple as a short list like this. Just make sure you talk through the entities with your interviewer to ensure you are on the same page. We'll introduce one more entity, a Generation, later once the deep dives actually need it, since it isn't something you'd naturally reach for this early.

Core Entities

- User
- Chat
- Message

Core Entities



As you move onto the design, your objective is to create a system that meets all functional and non-functional requirements. I recommend you start by satisfying the functional requirements and then layer in the non-functional requirements afterward. This keeps you focused and stops you from getting lost in the weeds.

API or System Interface

The API is the contract between the client and our system, and it's the bridge into the high-level design. We'll define one or two endpoints per functional requirement and keep moving.

First, a user starts a new chat. We use POST because we're creating a new Chat entity.

```
POST /chats -> { chatId }  
Body: {}
```



Next, the user sends a prompt and gets a response back. This is the one endpoint that isn't a plain request/response. The assistant message is streamed back token by token, and we return a `runId`, a handle for this in-flight response that the client uses to follow the stream. We use POST because we're creating a new Message on the server.

```
POST /chats/{chatId}/messages -> Message (streamed via SSE)
Body: {
  content
}
```



For the second functional requirement, we list a user's chats for the sidebar and load the messages for one chat. Both are GETs with cursor pagination, since a heavy user can have thousands of chats and a long chat can have thousands of messages.

```
GET /chats?cursor={cursor}&limit={n} -> Chat[]
GET /chats/{chatId}/messages?cursor={cursor}&limit={n} -> Message[]
```



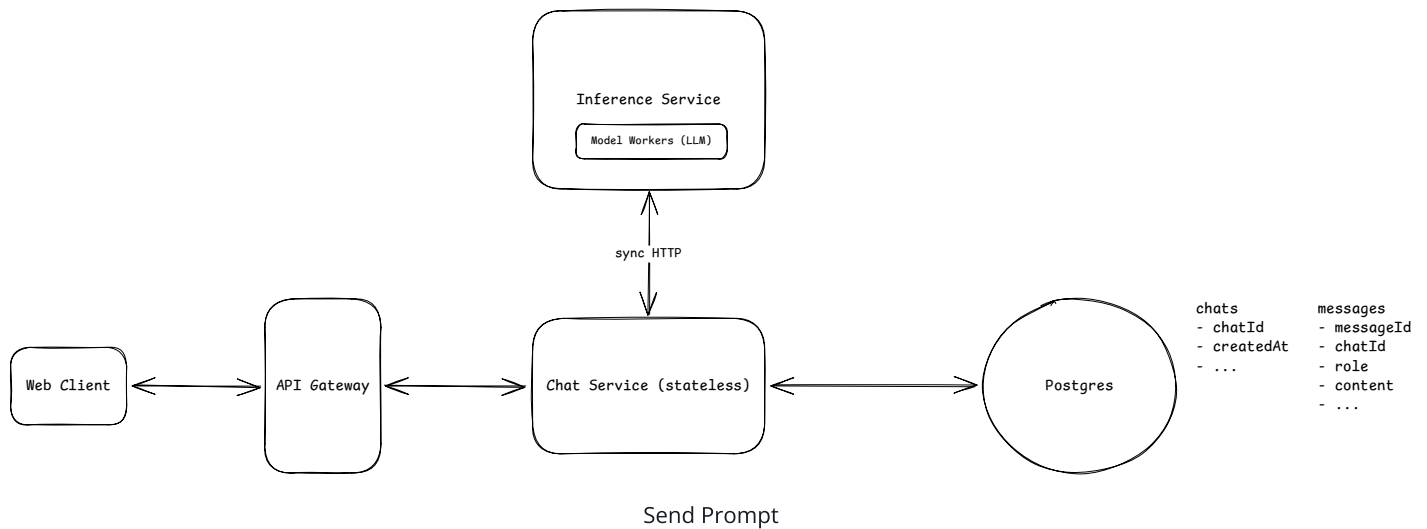
Notice the `userId` never shows up in a path or body. It comes from the session token or JWT, and chat ownership is checked server-side on every request. Passing `userId` in the body is a classic red flag, since anything the client sends can be forged. The streaming response uses SSE, which we'll justify in the deep dives, but the core CRUD surface is plain REST.

High-Level Design

We'll go one by one through the functional requirements. Both are short, and we're going to keep the design deliberately naive, with synchronous calls and no streaming or queues. The plan is to get a simple design that satisfies our functional requirements first, then layer on the complexity that satisfies our non-functional requirements through the deep dives. Starting with a working system and then breaking it is far better than starting with a "perfect" one that's hard to reason about.

1) Users should be able to send a prompt and receive an AI-generated response

When a user opens a chat, types a prompt, and hits enter, the client sends that prompt to our backend and eventually gets a response back. Let's lay out the minimum set of components to make that happen.



1. **Web Client:** The browser or mobile app where the user types prompts and reads responses. It's the chat UI.
2. **API Gateway:** The entry point for all client requests. It handles authentication, rate limiting, and routes requests to the right service.
3. **Chat Service:** A stateless service that owns chat and message persistence and orchestrates the call to the model. It's cheap to run and easy to scale horizontally, which matters because we'll want to scale it independently from the expensive inference layer.
4. **Postgres:** Our system of record, holding the `chats` and `messages` tables. The data is simple rows keyed by `chatId` and `userId`, so honestly almost any database would work here. We reach for Postgres as a sensible default rather than because anything about the problem demands a relational store.
5. **Inference Service:** Owns the GPU model workers that actually run the LLM. We treat the model itself as a black box that takes a prompt in and returns a completion. This is the expensive, GPU-bound part of the system, separated from the Chat Service so we can scale and schedule it on its own.

Here's how these interact when a user sends a prompt:

1. The user types a prompt, and the client sends a POST request to `/chats/{chatId}/messages`.
2. The API Gateway authenticates the request and forwards it to the Chat Service.
3. The Chat Service writes the user's message to the `messages` table.
4. The Chat Service makes a synchronous call to the Inference Service, which runs the prompt through the model and returns the full completion once it's done.

5. The Chat Service writes the assistant message back to the `messages` table and returns it to the client.

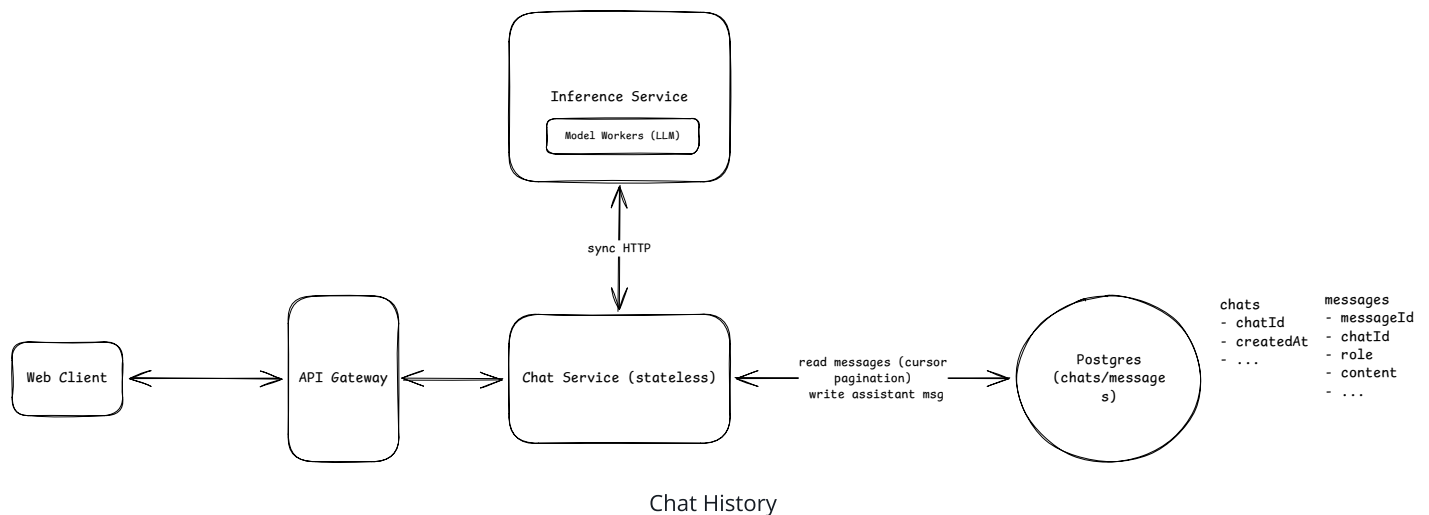
The split between Chat Service and Inference Service is the one design decision worth dwelling on here. The chat tier is cheap and stateless, while inference is GPU-bound and expensive, and since we'll want to scale the two independently, we separate them now.

Let me briefly acknowledge the elephant in the room. This is fully synchronous, so the client sits on that HTTP call until the entire response is generated, and a long response can take up to 30 seconds. That's 30 seconds of blank screen, which violates our TTFT requirement and feels broken. On top of that, the Chat Service is calling a GPU worker directly with no admission control (nothing deciding which requests to accept versus turn away when the workers are already saturated), which falls apart the moment GPUs become the bottleneck. We'll fix the first problem with streaming and the second with a scheduling layer, both in the deep dives. For now, it works.

2) Users should be able to view past chats and resume a conversation with context carried across turns

Users expect to come back tomorrow, scroll their old conversations, open one, and keep going as if the model remembers everything. Two things have to happen here, a read path for past chats and context carry-over on the next turn.

We don't need any new services for this. We add the read endpoints off the existing Postgres tables and a context-loading step inside the Chat Service.



For the read path:

1. `GET /chats` returns the user's chats ordered by recent activity, cursor-paginated for the sidebar.
2. `GET /chats/{chatId}/messages` returns one chat's messages, cursor-paginated so a long conversation doesn't load all at once.

For context carry-over, when the user sends a follow-up prompt on an existing chat:

1. The Chat Service queries the `messages` table for the prior messages in that chatId, ordered by creation time.
2. It builds the prompt by concatenating those messages (with their roles, user vs assistant) followed by the new user message.
3. It sends that combined prompt to the Inference Service, just like the first turn.
4. The new assistant message gets written back to the `messages` table, so the next turn can read it too.

This is the simplest thing that works. The model sees the whole conversation every turn, so it behaves like it remembers. But sending full history every turn has two obvious problems, since it breaks once a conversation grows past the model's context window and gets more expensive every turn as input tokens are billed per call. We'll tackle that with summarization and prefix caching in the deep dives.

That gets us a working system. It's simple, it satisfies both functional requirements, and it has exactly the bottlenecks our non-functional requirements warned us about. Let's go fix them.

Potential Deep Dives

With the functional requirements met, it's time to go back and earn the non-functional requirements. There's no single right set of deep dives here, or one correct order to tackle them in. In a real interview you'd have agreed on what matters with your interviewer when you outlined the non-functional requirements up front. These are the ones I'd expect to come up for this question, roughly in the order the design pushes you toward them, starting with streaming the response, then scheduling the GPUs, sharing them fairly, and keeping the cost from running away.

1) How do we stream tokens back fast, and keep the stream smooth?

Our synchronous design makes the user wait up to 30 seconds for a blank screen to turn into a full answer. The non-functional requirement asked for two things, a low time-to-first-token and a continuous, smooth flow after that, and those are two different requirements that fail in two different ways. The first is about how quickly the very first token reaches the screen, which is a pure latency problem. The second is about whether the rest of the stream arrives in order and without visible gaps, even as Chat Service instances come and go underneath us, which is a reliability problem. The mechanisms that solve them have almost nothing to do with each other, so we'll take them one at a time. First, how do we get that first token back fast?

⚡ Pattern: Real-time Updates

Streaming LLM tokens is a textbook **realtime updates** problem. The browser needs a live push channel from the server, and the backend needs a way to get each token from the worker that generated it over to whichever server is holding the user's connection. The same transport options (long-polling, SSE, WebSockets) and the same backend fanout show up in live comments, collaborative editing, and live dashboards.

Learn This Pattern

Bad Solution: Polling for Chunks



Good Solution: WebSockets



Great Solution: Server-Sent Events



SSE gets that first token onto the screen in milliseconds, which settles the responsiveness half of the requirement. But look closely at what we glossed over. We kept saying "the server holds the response open and writes tokens to it," as if one fixed server reliably sits between this user and the model for the full 30 seconds. Our Chat Service tier is stateless, horizontally scaled behind a load balancer, and redeployed all day long. The moment you take that seriously, two questions appear that the transport choice never touched. How does a token actually get from

the worker that produced it over to whichever Chat Service instance is holding this user's SSE connection right now? And what happens to the stream when that instance is replaced mid-generation?

This is the second half of the requirement, keeping the stream smooth and unbroken from the first token to the last.

Bad Solution: Pin the Connection to One Server



Good Solution: Decouple With Redis Pub/Sub

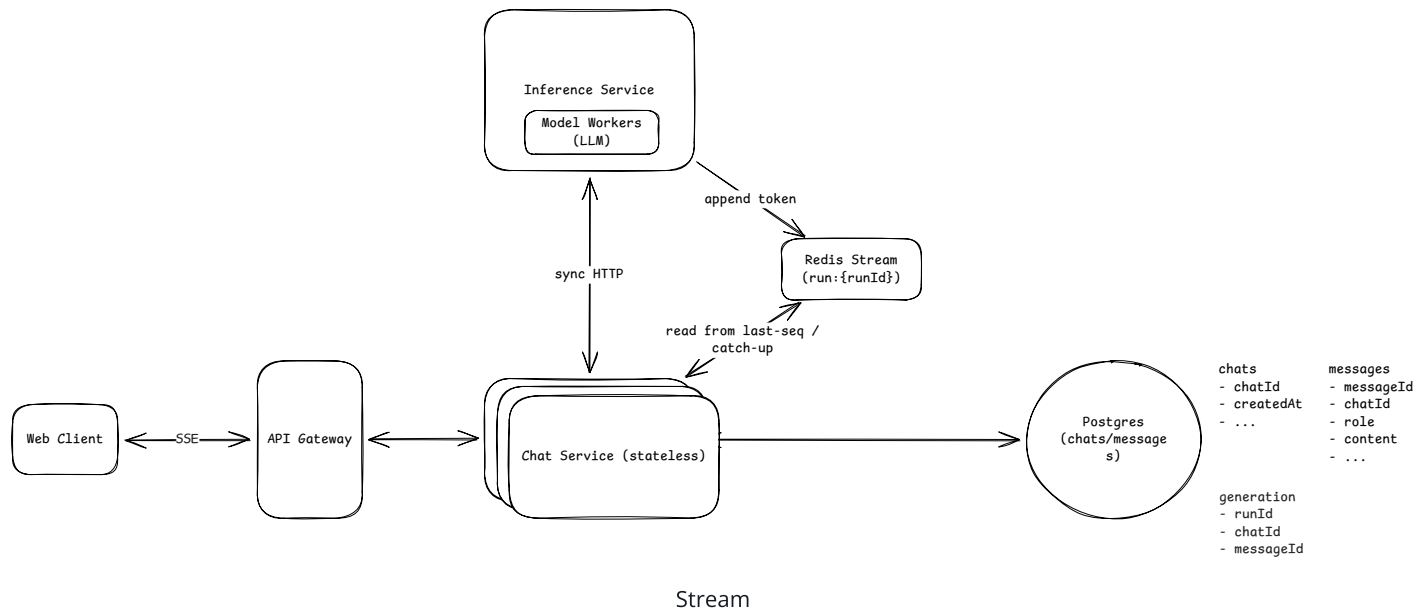


Great Solution: Decouple With Redis Streams and Replay



After applying both "Great" solutions, here's the full flow on a send:

1. The Chat Service persists the user's message and mints a fresh `runId`, returning it to the client right away.
2. The Chat Service hands the assembled prompt and that `runId` off to the Inference Service, which lines up a worker to start generating (we'll put a queue here in the next deep dive).
3. The client opens an SSE connection for that `runId`, and the load balancer routes it to any Chat Service instance.
4. That Chat Service instance starts a blocking `XREAD` on the `runId` stream, from the beginning for a fresh connection or from the client's last-seen entry ID on a reconnect.
5. The inference worker generates tokens and `XADD`s each one to the `runId` stream.
6. The Chat Service instance reads new entries, forwards them down the SSE connection, and the browser appends them. If the connection drops, the browser reconnects, lands on some Chat Service instance, and that instance replays from the last-seen ID before continuing live.



That `runId` we keep minting deserves to be a first-class entity. A **Generation** is a single inference attempt for a message, carrying the `runId`, the `chatId` and `messageId` it belongs to, a status that moves through queued, streaming, and then done, cancelled, or failed, the model that served it, and the input and output token counts we'll lean on for billing and quotas. This is the extra entity we flagged back at the entities stage and deferred, since nothing in the basic request path needed it. Now that a run has its own lifecycle and its own stream, it's earned its place, and the scheduling and cancellation work still ahead is really a set of operations on a Generation rather than on a message.

2) How do we route and schedule generation requests across GPU workers?

GPUs are the bottleneck, full stop. They're the most expensive resource in the system and the one in shortest supply, so how we route work to them decides both our cost and our latency under load. It's worth pausing on just how expensive. A frontier model is far too big to fit on a single GPU, so its weights get split across a whole box of them, and serving 120k concurrent streams means standing up thousands of those boxes. That puts you at tens of thousands of GPUs for this one model, and the labs running systems at this scale spend staggering amounts on compute, easily hundreds of millions to billions of dollars a year. When the hardware costs that much, every percentage point of utilization you leave on the table is real money, which is exactly what makes the scheduling decisions in this section worth the effort.



Reality check. For a real-world anchor, OpenAI's reported inference compute ran around \$1.8B in 2024 and has climbed into the multiple billions since. You don't need the exact

figure in an interview, but the order of magnitude is the whole point, since a bill that size is what justifies all the engineering effort we're about to spend squeezing more out of each GPU.

🕒 Pattern: Managing Long Running Tasks

A single generation can run for 30 seconds on a scarce, expensive worker. That's the **long-running tasks** pattern, where instead of tying up the request thread waiting, you hand the work to a pool of workers through a queue and let them pull when they have capacity. The same queue-plus-worker-pool shape shows up in video transcoding, batch ML jobs, and any system where the unit of work is too heavy to do inline.

Learn This Pattern

Bad Solution: Direct Synchronous Dispatch



Good Solution: Queued Worker Pool



Great Solution: Queue Plus Continuous Batching and Backpressure



Why batching wins, in one analogy. A GPU keeps the model's weights resident in its high-bandwidth memory, but the compute units can't do math on them there. They work out of a tiny pool of on-chip memory that's nowhere near big enough to hold billions of weights, so every forward pass has to stream the entire weight set from high-bandwidth memory through the compute units just to produce a token. Picture the weights as parts in a warehouse and the compute as a tiny workbench. The parts never leave the building, but to build anything you still have to haul the full set from the warehouse over to the bench. For a single token you make that whole haul and then do a trivial amount of assembly, one vector's worth of math, so the bench sits idle most of the time waiting on

[Show More](#) 

3) How do we keep heavy users from monopolizing GPUs while giving paid tiers a better experience?

This is a multi-tenant system sharing one scarce GPU pool, and the costs are wildly uneven. One user firing a 30k-token prompt burns far more compute than a hundred users sending one-liners. We need fairness across users so nobody can starve everyone else, and we need business priority across tiers so paying customers get a better experience when things are tight. A flat requests-per-minute cap can't express either of those.

Bad Solution: Flat Global Rate Limit



Good Solution: Per-User Limits



Great Solution: Cost-Aware Quotas with Tier Priority and Graceful Degradation



4) As conversations get longer, how do we control inference cost without making the assistant feel forgetful?

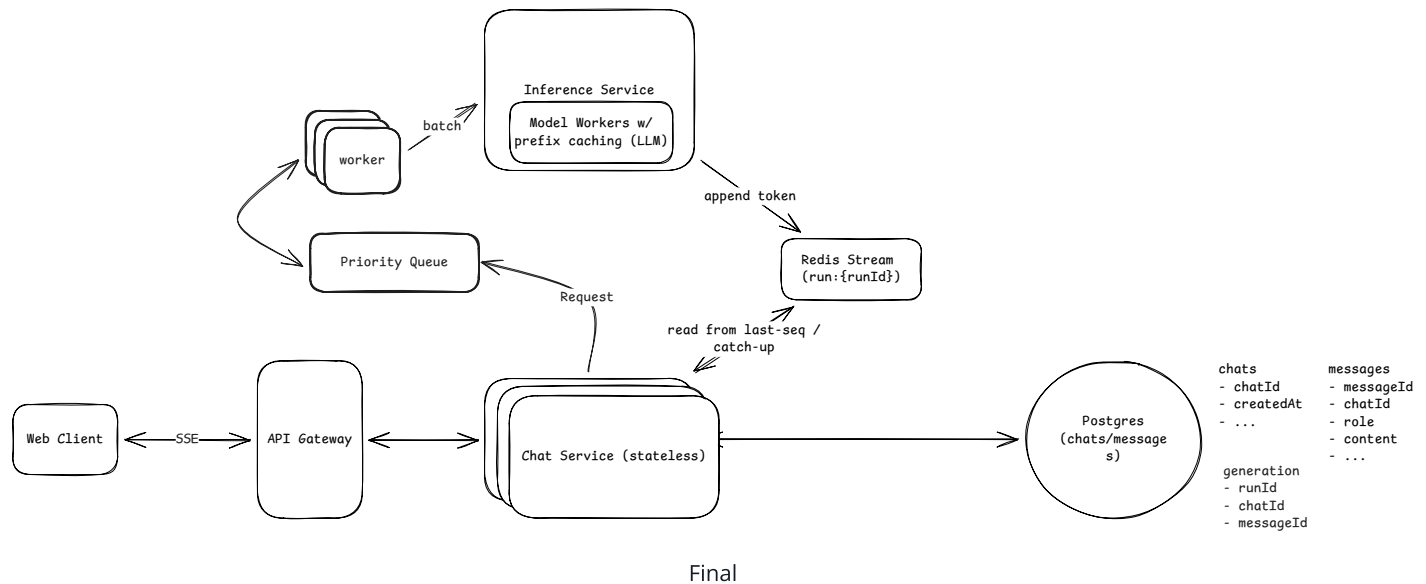
Recall our high-level design replays the entire conversation into the model on every turn. That's the simplest thing that works, and it's also the expensive mistake, so rather than dwell on it as its own option we'll treat it as the baseline we're fixing. A 50-turn chat at ~500 tokens per turn means we're shipping ~25k input tokens on the next prompt, and since input tokens are billed per call, cost and latency climb with every single turn. Worse, it has a hard ceiling. Once the conversation grows past the model's context window the request simply can't fit. Real assistants usually just surface this, telling you the chat has gotten too long rather than failing silently, but leaning on that as your only answer means the product quietly stops working for exactly the power users who chat the most. It's fine for a five-turn chat and untenable as a general approach. What we want is to keep the assistant feeling like it remembers without paying to re-read the whole transcript each time.

Good Solution: Truncation**Great Solution: Prefix Caching and a Rolling Summary****Cancelling A Run And Reclaiming The GPU**

The one operation on a Generation we haven't shown yet is cancellation, and it's the clearest payoff for having made the run first-class. When the user hits stop, the client makes a plain HTTP call to `POST /chats/{chatId}/runs/{runId}/cancel`, exactly the side-channel we set aside when we chose SSE. The Chat Service flips that Generation's status to `cancelled` and publishes a cancel signal on a control channel keyed by the `runId`. The worker checks that channel between token batches, and the moment it sees the signal it drops the sequence and stops generating. This matters more than it first sounds. A cancelled 30-second generation that keeps running is pure wasted GPU, and GPU is the scarcest, most expensive thing in the whole system, so reclaiming it the instant the user stops caring is real money back.

Worth being clear that closing the tab is not a cancel. We built the Redis Stream and SSE reconnect precisely so a dropped connection isn't read as the end of a run, and like ChatGPT we keep generating in the background. The user can reopen the chat and reconnect to the stream, or just refetch the finished message from Postgres once it's done. Cancellation has to be an explicit signal from the user, never an accident of the network.

After applying the "great" solutions, the design has grown from the naive synchronous version into something that streams fast and stays smooth, schedules GPUs efficiently, shares them fairly, and keeps cost in check. Here's roughly how it all fits together:



Some additional deep dives you might consider

There's plenty we couldn't fit here. A few more directions worth thinking through on your own:

1. **Safety and moderation:** We put content moderation below the line to keep the focus on serving, but plenty of interviewers will want to see it. The usual shape is a cheap classifier on the prompt on the way in, and a second pass on the output as it streams. The output pass is where it gets tricky, because you've usually already streamed some tokens to the user by the time the check trips, so you have to decide whether to moderate in chunks before flushing each one or pull the message back after the fact.
2. **Why one model needs a whole box of GPUs:** We said the weights get split across a whole box of GPUs without saying how. A frontier model's weights don't fit in a single GPU's memory, so you split them across GPUs, either with tensor parallelism where each GPU holds a slice of every layer, or pipeline parallelism where each GPU holds whole layers and hands off to the next. Both lean on fast interconnects, NVLink between GPUs in a box and InfiniBand across boxes, and that interconnect can become its own bottleneck.
3. **Speculative decoding:** For some interviews, especially at the senior and staff level, you'll go much deeper into inference internals, and this is one of the levers worth knowing. Because decode runs one token at a time, a small cheap "draft" model can guess the next few tokens and the big model verifies all of them in a single forward pass. When the draft guesses right you get several tokens for the cost of one step, a real win for both time-to-first-token and throughput.

4. **Cheaper requests through routing and caching:** Not every prompt needs the biggest model. Routing simple queries to a smaller model, and caching responses for semantically similar prompts (keyed off an embedding rather than the exact string), both cut cost without the user noticing. This pairs naturally with the tiered fairness deep dive, where free traffic is the first to get routed down.
5. **Multimodal input and cross-chat memory:** We scoped to text in, text out. Real ChatGPT also takes images and audio, which changes tokenization and bloats the prefill, and it remembers facts about you across separate conversations. That cross-chat memory is a retrieval problem, embedding past messages and pulling the relevant ones into the prompt, rather than the single-conversation summarization we did here.

What is Expected at Each Level?

Ok, that was a lot. You may be thinking, "how much of that is actually required from me in an interview?"

Mid-level

For this question, a mid-level candidate will have clearly defined the API endpoints and data model and landed on a working synchronous high-level design that handles sending a prompt and viewing past chats with context carried across turns. I want to see them recognize that a 30-second blank screen won't fly and reach for a push-based streaming model like SSE, even if it takes some prompting. They should understand that GPUs are the bottleneck and at least propose putting a queue in front of the workers, though they may not get to continuous batching or backpressure on their own.

Senior

For this question, senior candidates are expected to speed through the high-level design so they can spend time on at least 2 of the streaming fanout, GPU scheduling, and fairness deep dives in detail. You should be able to articulate the SSE-vs-WebSocket choice from the one-way nature of token streaming, and the queue-plus-continuous-batching tradeoff for GPU utilization. I also expect a senior candidate to recognize that cost grows with conversation

length and to propose summarization or truncation, even if they don't reach prefix caching unaided.

Staff+

For a staff+ candidate, expectations are high regarding depth and quality of solutions, particularly for the complex scenarios discussed above. Great candidates drive 3+ of the deep dives with real depth and bring the GPU economics into the conversation unprompted, the back-of-envelope that ~120k concurrent streams means tens of thousands of GPUs and a seven-figure daily bill, which is what justifies continuous batching and backpressure in the first place. They reach prefix caching for context cost on their own and cleanly separate fairness-across-users (cost-aware per-user budgets) from priority-across-tiers (tier-weighted queueing), which is a distinction weaker candidates blur. The hallmark is insight beyond the textbook, where a staff+ candidate leaves the interviewer understanding something new about serving LLMs at scale, whether that's continuous batching, KV-cache reuse, or how degradation should fall on free traffic first.

Worth calling out that at this level interviewers often expect the more esoteric inference trivia too, things like speculative decoding. It's become widespread enough that a practicing engineer is assumed to have picked it up, so it's past the "curiosity" threshold and doesn't get the pass that, say, not knowing geohashes might get you just because you've never worked on that kind of problem.

Test Your Knowledge

Take a quick 15 question quiz to test what you've learned and identify gaps.

 Start Quiz